

Running the simulator without ROS2

Overview

Panisim provides a standalone mode that allows you to run simulations without requiring the ROS2 middleware. This lightweight mode is ideal for quick testing, development, and scenarios where the full ROS2 ecosystem is not needed.

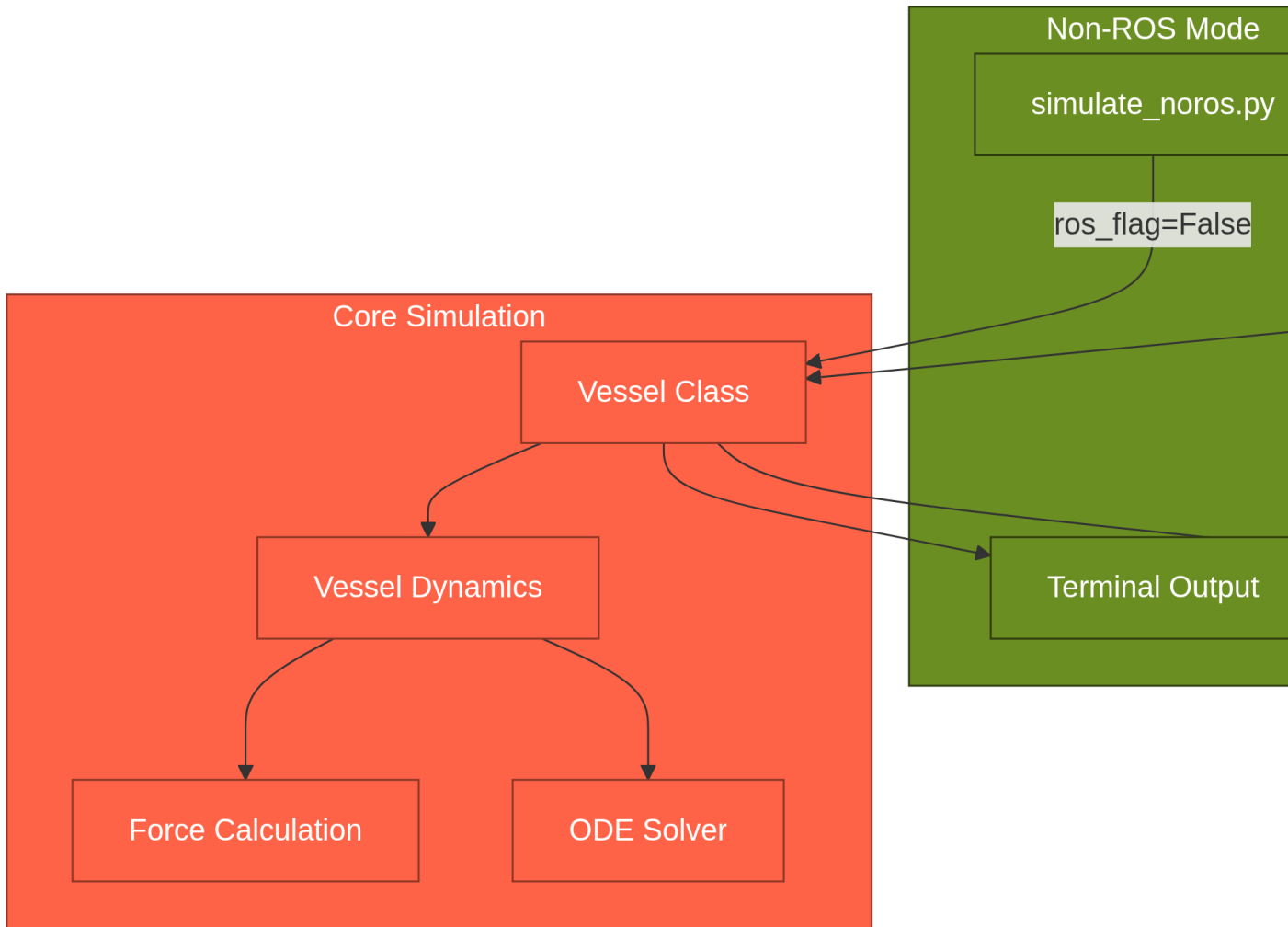
The non-ROS mode:

- Eliminates ROS2 dependencies and overhead
- Provides terminal-based feedback
- Supports the full vessel dynamics model
- Allows custom controller implementation
- Simplifies debugging of core simulation components

Implementation Details

The non-ROS mode is implemented through two key components:

1. **`simulate_noros.py`**: A standalone script that initializes and runs the simulation
2. **`class_vessel.py`**: The core vessel class with a `ros_flag` parameter that controls ROS-specific behavior



The `ros_flag` Parameter

The `ros_flag` parameter in the `Vessel` class constructor determines whether the vessel should use ROS2-specific functionality:

```
def __init__(self, vessel_params: Dict, hydrodynamic_data: Dict, vessel_id: int, ros_flag: bool):
    """Initialize vessel with parameters and hydrodynamic data.

    Args:
        vessel_params: Dictionary containing vessel parameters
        hydrodynamic_data: Dictionary containing hydrodynamic coefficients
        vessel_id: Unique identifier for the vessel
        ros_flag: Flag indicating whether to use ROS2 functionality
    """
    self.ros_flag = ros_flag
    # ... rest of initialization
```

When `ros_flag` is set to `False`:

1. No ROS2 node is created for the vessel
2. No publishers or subscribers are established
3. Command inputs must be set directly in code or through custom control logic
4. Simulation data is displayed in the terminal instead of through ROS2 topics

Running the Non-ROS Simulator

To run Panisim without ROS2:

1. Navigate to the `makara` directory
2. Run the `simulate_noros.py` script:

```
cd /workspaces/mavlab/ros2_ws/src/mav_simulator/mav_simulator
python3 simulate_noros.py
```

The simulation will use the same configuration files as the ROS2 version, loading vessel parameters, hydrodynamics, and initial conditions from the input files.

Simulation Output

The non-ROS simulator provides detailed terminal output showing:

- Simulation time
- Vessel position and orientation
- Linear and angular velocities
- Forces acting on the vessel, including:

- Hydrodynamic forces
 - Gravitational forces
 - Control surface forces
 - Thruster forces
- Control surface angles
 - Thruster RPMs

Example output:

Starting simulation...

Time [s] | Position [x, y, z] | Velocity [u, v, w]

Time: 0.00

Position [x, y, z]: [0.00, 0.00, 0.00]

Velocity [u, v, w]: [0.50, 0.00, 0.00]

Orientation [phi, theta, psi] (deg): [0.00, 0.00, 0.00]

Hydrodynamic forces: [-0.40, 0.00, 0.00, 0.00, 0.00, 0.00]

Gravitational forces: [0.00, 0.00, 0.00, 0.00, 0.00, 0.00]

Control surface forces: [0.00, 0.00, 0.00, 0.00, 0.00, 0.00]

Thruster forces: [0.00, 0.00, 0.00, 0.00, 0.00, 0.00]

Control surface angles [deg]: []

Thruster RPMs: []

Time: 0.01

Position [x, y, z]: [0.01, 0.00, 0.00]

Velocity [u, v, w]: [0.50, 0.00, 0.00]

...

Implementing Custom Controllers

One of the key advantages of the non-ROS mode is the ability to easily implement custom controllers directly in the code. The `vessel_ode` method in `class_vessel.py` contains a section specifically for non-ROS mode:

```
if not self.ros_flag:
    ## TODO: Implement your controller logic here to get the actuator commands

    ## example
    # if self.control_surface_control_type == 'fixed_rudder':
    #     self.delta_c = con.fixed_rudder(t, state, n_control_surfaces, 10.0) #Enter rudder a
```

```

# elif self.control_surface_control_type == 'switching_rudder':
#     self.delta_c = con.switching_rudder(t, state, n_control_surfaces)
# else:
#     raise ValueError(f"Invalid control surface control type: {self.control_surface_control_type}")

# # Get thruster commands
# if self.thruster_control_type == 'fixed_rpm':
#     self.n_c = con.fixed_thrust(t, state, n_thrusters, 1000.0) #Enter RPM here
# else:
#     raise ValueError(f"Invalid thruster control type: {self.thruster_control_type}")
pass

```

To implement your own controller:

1. Create a controller module (e.g., `my_controller.py`) with your control algorithms
2. Import your controller module in `simulate_noros.py`
3. Modify the `vessel_ode` method to use your controller:

```

# In vessel_ode method of class_vessel.py
if not self.ros_flag:
    # Custom controller implementation
    if hasattr(self, 'control_surface_control_type'):
        if self.control_surface_control_type == 'my_controller':
            self.delta_c = my_controller.calculate_rudder(t, state, self.n_control_surfaces)

```

Example: Adding a Simple PID Controller

Here's an example of how to add a simple PID heading controller:

1. Create a controller module:

```

# my_controller.py
import numpy as np

def pid_heading_controller(t, state, n_surfaces, desired_heading=0.0):
    """Simple PID heading controller.

    Args:
        t: Current time
        state: Vessel state vector
        n_surfaces: Number of control surfaces
    """

```

```

        desired_heading: Target heading in radians

Returns:
    numpy.ndarray: Control surface angles in radians
    """
    # Extract current heading
    current_heading = state[11] # psi is at index 11

    # Calculate heading error
    heading_error = desired_heading - current_heading

    # Normalize to -pi to pi
    heading_error = (heading_error + np.pi) % (2 * np.pi) - np.pi

    # PID gains
    Kp = 0.5
    Ki = 0.01
    Kd = 0.1

    # Calculate PID terms (simplified - would need integral and derivative state)
    p_term = Kp * heading_error

    # Calculate rudder command (assume first surface is main rudder)
    rudder_angle = np.clip(p_term, -np.pi/4, np.pi/4)

    # Create control surface command array
    delta_c = np.zeros(n_surfaces)
    if n_surfaces > 0:
        delta_c[0] = rudder_angle

    return delta_c

```

2. Modify the `vessel_ode` method in `class_vessel.py`:

```

if not self.ros_flag:
    # Import controller at the top of the file
    import my_controller

    # Use controller for rudder commands
    self.delta_c = my_controller.pid_heading_controller(
        t, state, self.n_control_surfaces, desired_heading=np.pi/4) # 45 degrees

```

```
# Set thruster commands if needed
if self.n_thrusters > 0:
    self.n_c = np.ones(self.n_thrusters) * 1000.0 # Fixed RPM
```

Customizing the Non-ROS Simulator

Modifying Simulation Parameters

You can customize the simulation by modifying the `simulate_noros.py` file:

1. Change input file path:

```
sim_params, agents = read_input("/path/to/your/custom_input.yml")
```

2. Modify output format:

```
# Customize the print statements in the simulation loop
print(f"Custom format: time={vessel.t:6.2f}, position=[{pos[0]:6.2f}, {pos[1]:6.2f}]"
```

3. Add data logging:

```
# Add at the beginning of the simulate function
import csv
log_file = open('simulation_log.csv', 'w', newline='')
csv_writer = csv.writer(log_file)
csv_writer.writerow(['Time', 'X', 'Y', 'Z', 'U', 'V', 'W', 'P', 'Q', 'R', 'Phi', 'Theta'])

# Inside the simulation loop
csv_writer.writerow([vessel.t, pos[0], pos[1], pos[2], vel[0], vel[1], vel[2],
                    vessel.current_state[3], vessel.current_state[4], vessel.current_state[5],
                    angles[0], angles[1], angles[2]])
```

Extending the Simulator

You can extend the non-ROS simulator for specific use cases:

1. Batch simulations:

```
def run_batch_simulations():
    results = []
    for initial_speed in [0.5, 1.0, 1.5, 2.0]:
        sim_params, agents = read_input("/workspaces/mavlab/inputs/simulation_input.yml")
        agents[0]['vessel_config']['initial_conditions']['start_velocity'][0] = initial_speed
```

```

        vessel = Vessel(agents[0]['vessel_config'], agents[0]['hydrodynamics'], vessel_i)
        vessel.simulate()
        # Extract results
        final_pos = vessel.history[-1, 6:9]
        results.append((initial_speed, final_pos))
    return results

```

2. Parameter studies:

```

def parameter_study():
    import matplotlib.pyplot as plt

    turning_radii = []
    rudder_angles = np.arange(-30, 31, 5) * np.pi / 180 # -30 to 30 degrees

    for rudder_angle in rudder_angles:
        # Set up simulation with fixed rudder angle
        sim_params, agents = read_input("/workspaces/mavlab/inputs/simulation_input.yml")
        vessel = Vessel(agents[0]['vessel_config'], agents[0]['hydrodynamics'], vessel_i)

        # Custom controller for fixed rudder
        vessel.control_surface_control_type = 'fixed'
        vessel.fixed_rudder_angle = rudder_angle

        # Run simulation
        vessel.simulate()

        # Calculate turning radius from final path
        # (simplified - would need actual path analysis)
        path = vessel.history[:, 6:8] # x, y positions
        turning_radius = calculate_turning_radius(path)
        turning_radii.append(turning_radius)

    # Plot results
    plt.plot(rudder_angles * 180 / np.pi, turning_radii)
    plt.xlabel('Rudder Angle (degrees)')
    plt.ylabel('Turning Radius (m)')
    plt.savefig('turning_radius_study.png')

```


Advantages of Non-ROS Mode

The non-ROS mode offers several advantages:

1. **Simplicity:** No need to set up ROS2 environment or handle ROS-specific configurations
2. **Performance:** Lower overhead without ROS middleware, especially for rapid prototyping
3. **Debugging:** Easier to debug core simulation components without ROS complexity
4. **Quick Testing:** Fast iteration for testing vessel dynamics or controller concepts
5. **Portability:** Can run anywhere Python is available, without ROS2 installation
6. **Learning:** Excellent starting point for understanding vessel dynamics before moving to ROS2
7. **Batch Processing:** Efficient for running multiple simulations for parameter studies or optimization

Limitations

Compared to the ROS2 mode, the non-ROS mode has some limitations:

1. **No Visualization:** Lacks the web-based visualization dashboard available in ROS2 mode
2. **Limited External Interaction:** No standardized way to interact with external systems
3. **No Distributed Simulation:** Cannot distribute components across multiple processes/machines
4. **No Standardized Messaging:** No access to ROS2's message types and communication patterns
5. **Manual Data Collection:** Requires custom code for data logging and analysis

Conclusion

The non-ROS mode provides a lightweight alternative for running Panisim simulations without the complexity of the ROS2 ecosystem. It's particularly useful for rapid development, testing, and educational purposes. By understanding the implementation through `ros_flag` and custom controller integration, users can leverage this mode for a wide range of applications while still benefiting from Panisim's accurate vessel dynamics modeling.